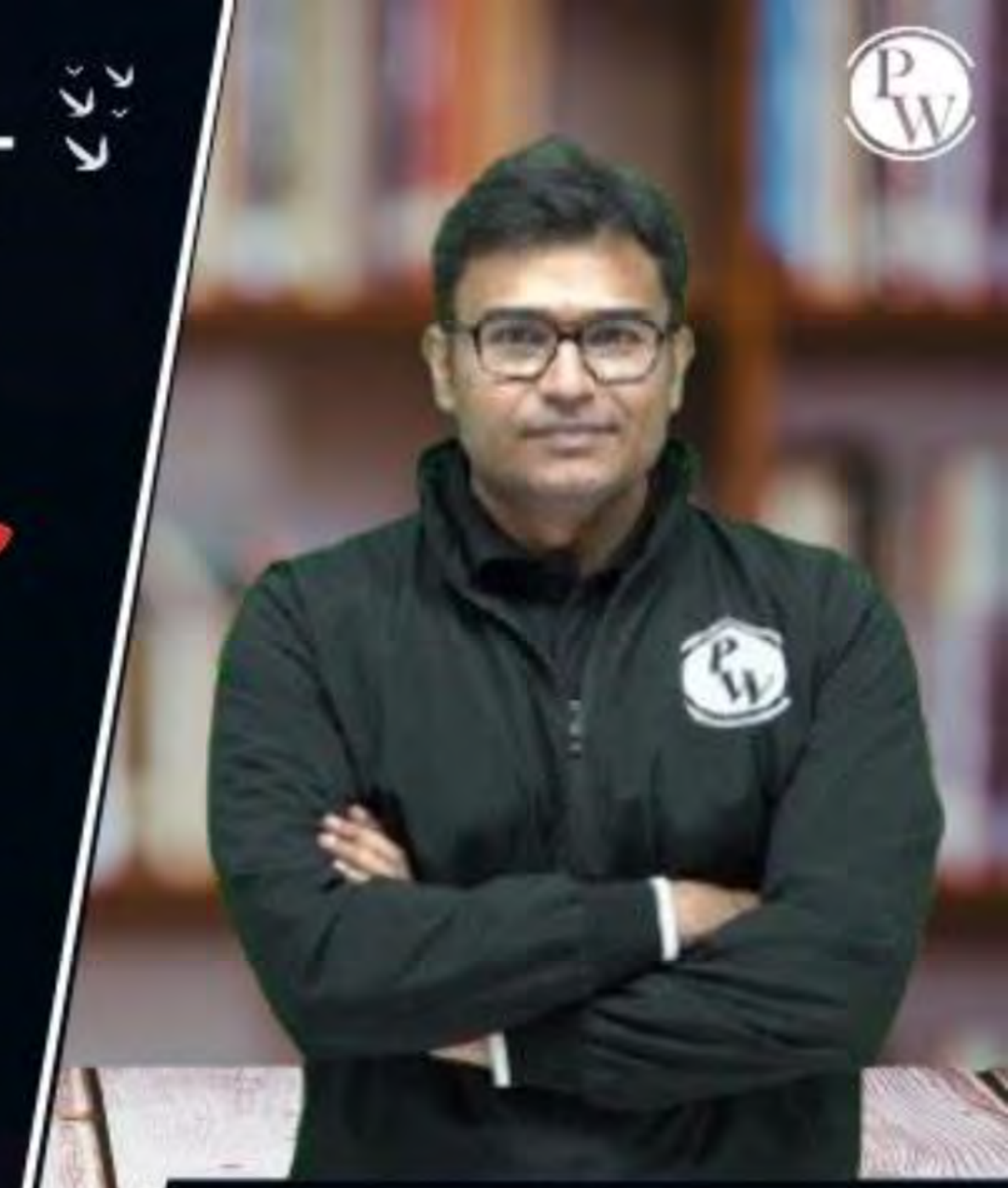
Computer Science & IT

Theory of Computation



TM

Lecture No. 1



By- Ravindra Sir

Recap of Previous Lecture



- Topic
- Topic
- Topic

CFL, PYQs

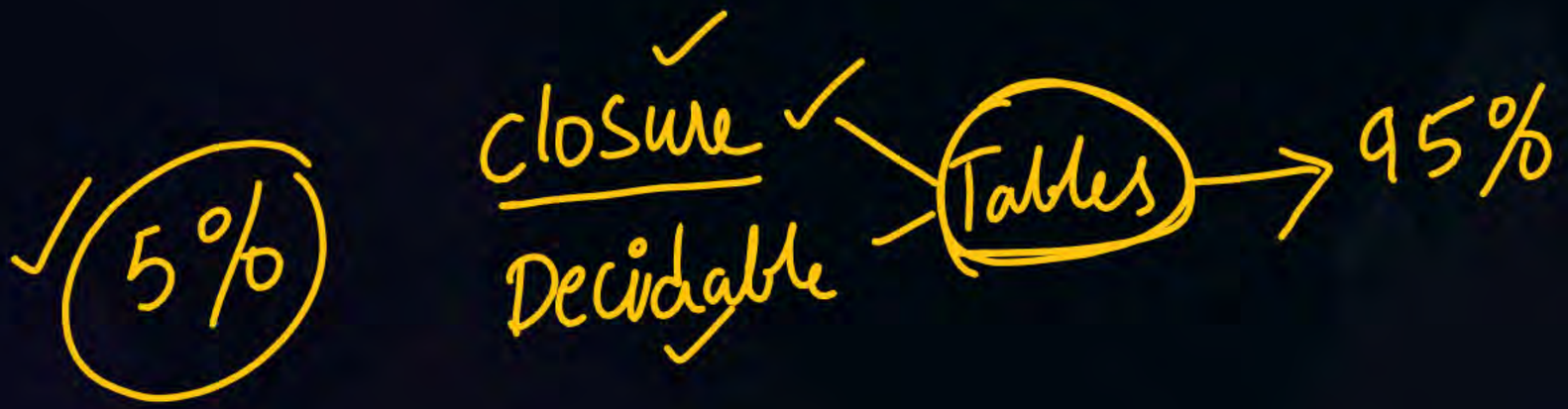
Topics to be Covered



- Topic
- Topic
- Topic

TM

Rec, RE, CSL, and TM



Rec, RE, CSL, TM

Turing machines

1. Turing machine theory
2. TM $\longrightarrow$ L(TM)
3. L $\longrightarrow$ TM
4. TM as a transducer
   $f(x,y) = x+y$
5) church turing thesis
6) Variations of TM
   No Variation has more power than Standard TM

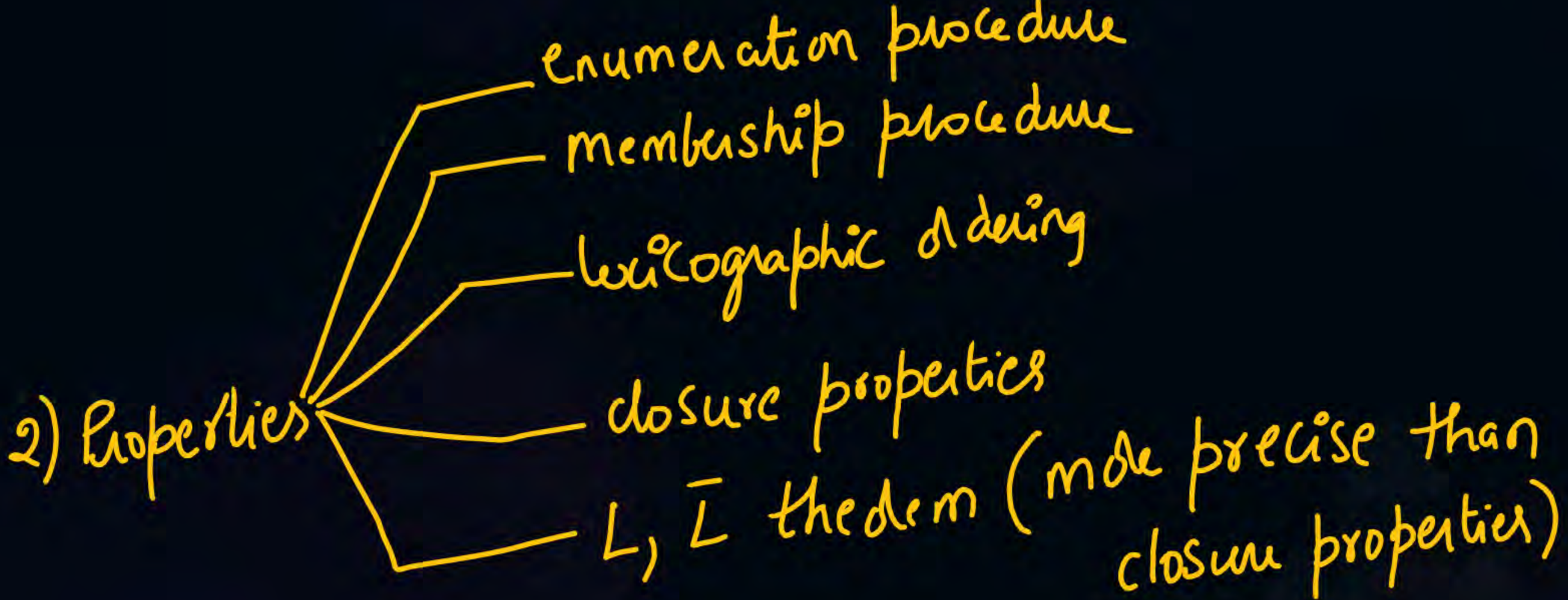
Rec and RE languages:-

1. Definition

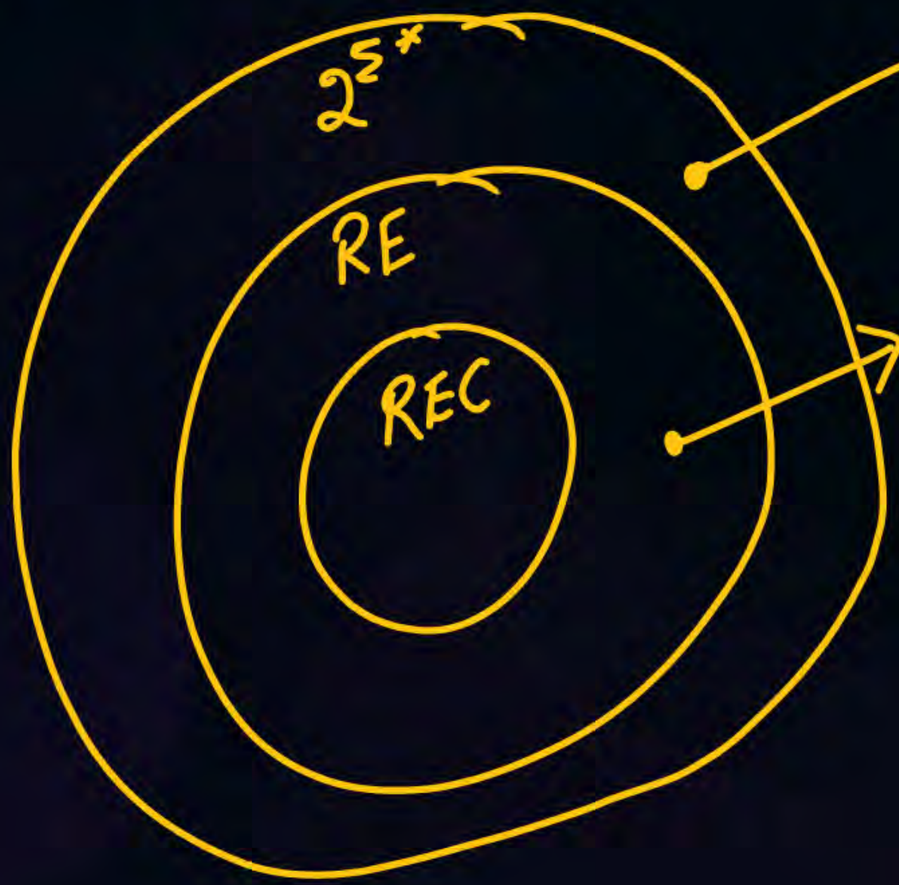


1) Recursive
2) RE but not recursive
1 + 2) = RE
3) Not RE

– 3 – 2
– 1


2) Properties
enumeration procedure
membership procedure
lexicographic ordering
closure properties
L, $\bar{L}$ theorem (more precise than closure properties)

3) RE but not REC, not RE



L_d → diagonalization language of TM

L_u → universal language of Turing machines

4) undecidable problems in TM



(we will add 6 more problem)

5) Rice's theorem → applicable only to RE languages

Turing machine theory

$M(Q, \Sigma, \Gamma, \delta, q_0, B, F)$ $\delta \rightarrow$ Transition function

$Q \rightarrow$ Finite set of states

$\Sigma \rightarrow$ i/p alphabet

$\Gamma \rightarrow$ Tape alphabet

$q_0 \rightarrow$ initial state

$B \rightarrow$ Blank ($\square$)

$F \rightarrow$ Set of final states

$\Gamma \supseteq \Sigma$

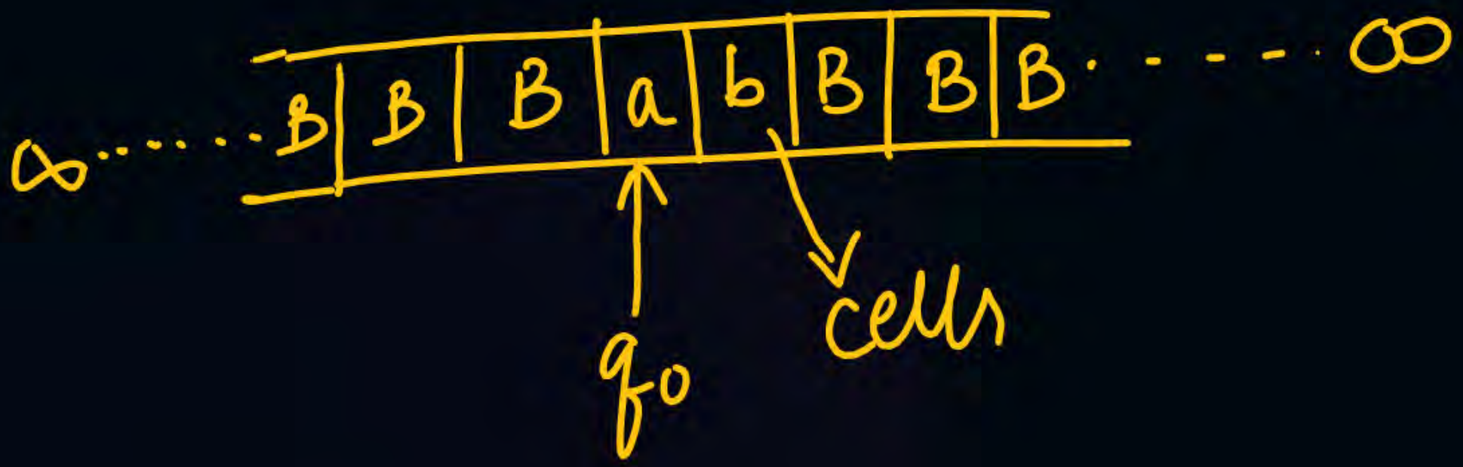
$\Sigma = \{a, b\}$

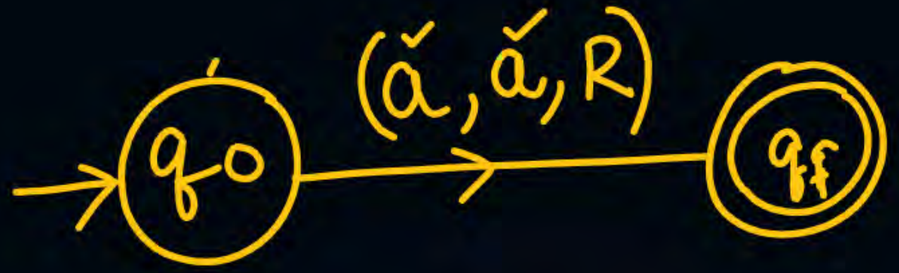
$\Gamma = \{a, b, 0, 1\}$

DTM: $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ ✓

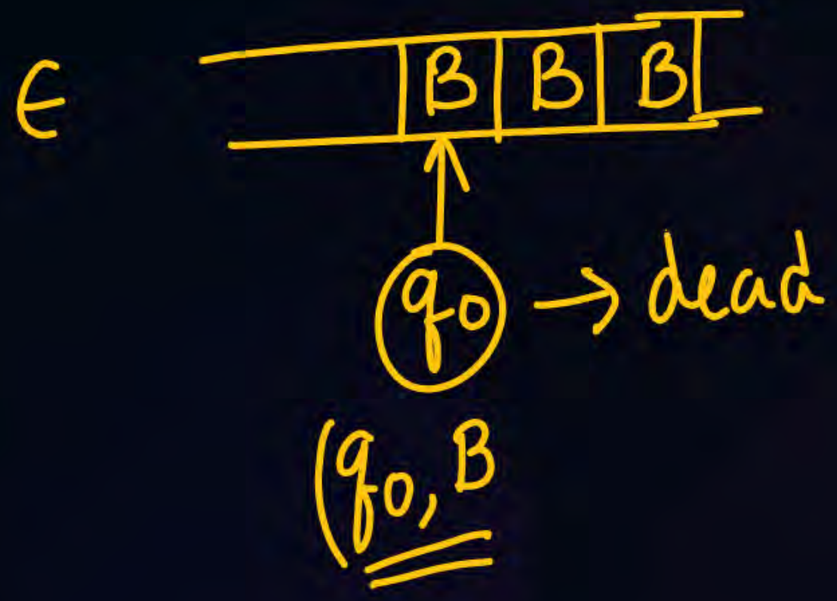
NTM: $\delta : Q \times \Gamma \rightarrow 2^{Q \times \Gamma \times \{L, R\}}$

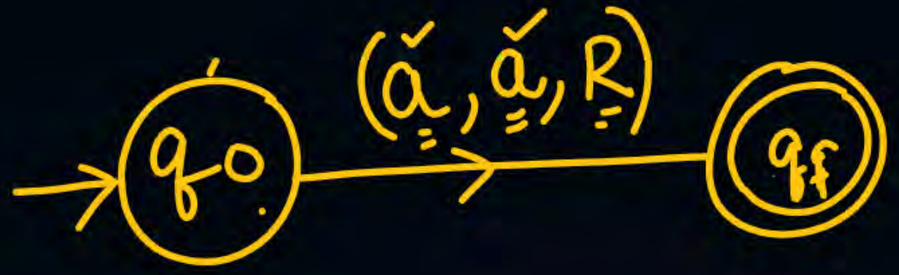
ab



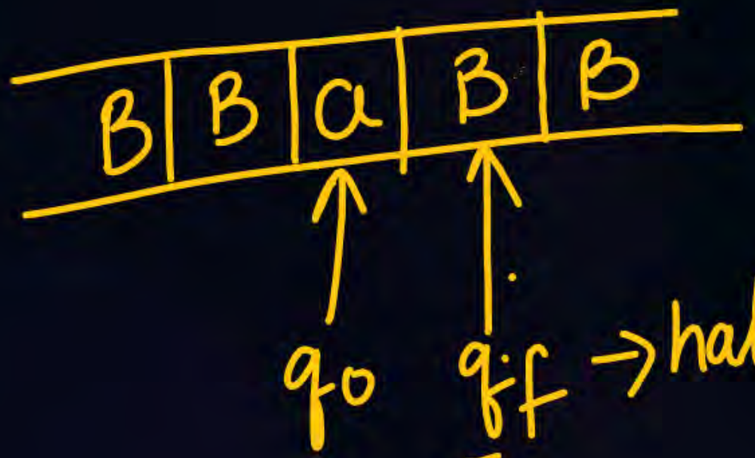


$\epsilon \rightarrow$ halt, Reject



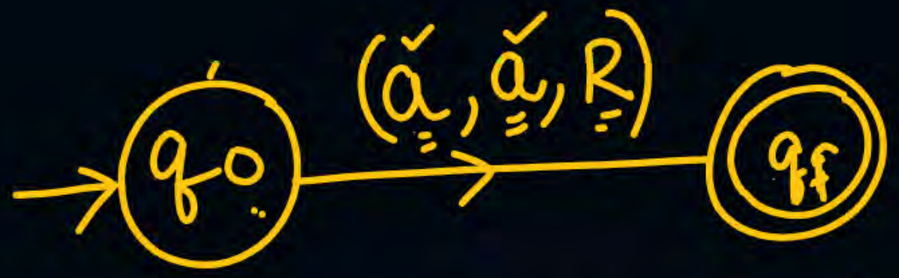


ϵ



$\epsilon \rightarrow$ halt, Reject

$a \rightarrow$ halt, accept



ϵ

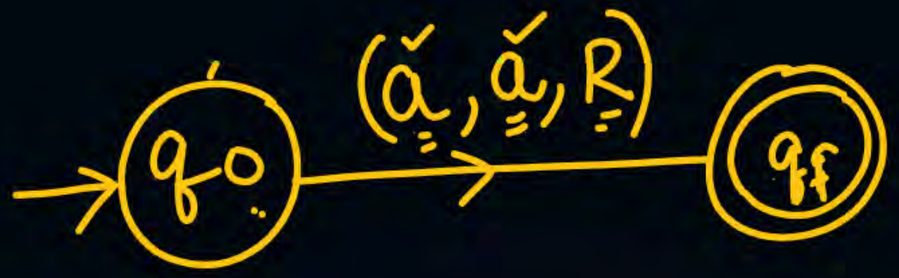
B B b B B

q0 → halt

ϵ → halt, Reject

a → halt, accept

b → halt, rejected



ϵ B B a a B B

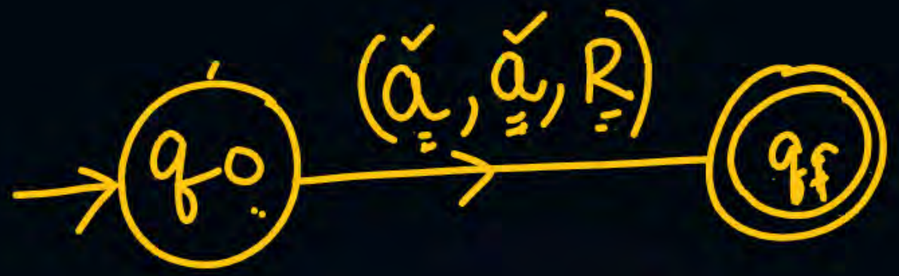
q0 qf → halt

ϵ → halt, Reject

a → halt, accept

b → halt, rejected

aa → halt, acc



ϵ B B a b B B

q0 qf →

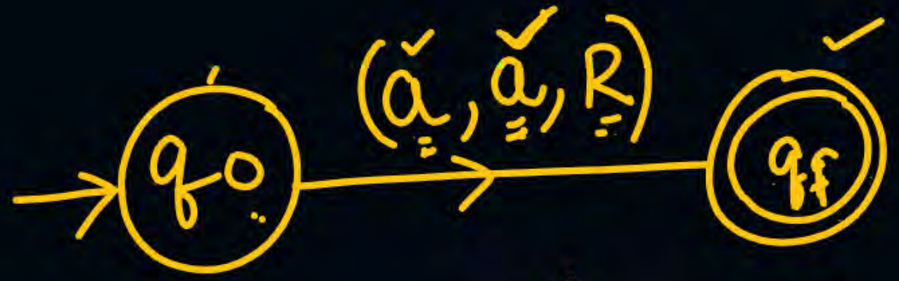
ϵ → halt, Reject

✓a → halt, accept

b → halt, rejected

✓aa → halt, acc

✓ab → halt, acc



B B a a a B B

q_0 q_f → halt

ϵ → halt, Reject

✓a → halt, accept

b → halt, rejected

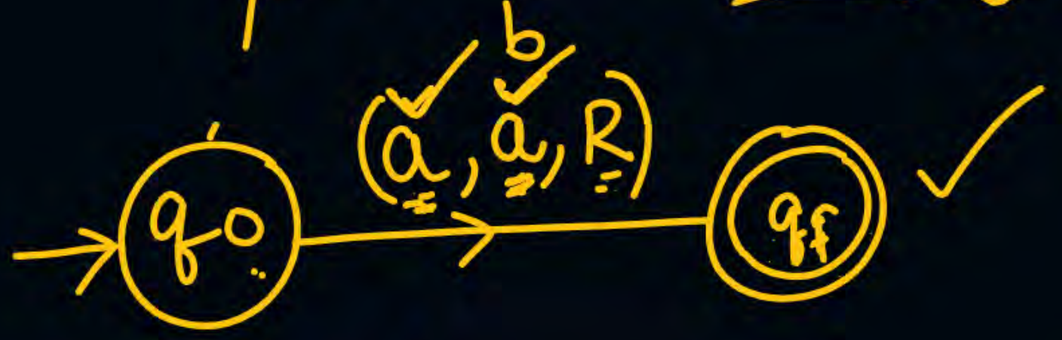
✓aa → halt, acc

✓ab → halt, acc

aaa → halt, acc

Changing batteries

(a, b, R)



B B a a a B B

q0 qf → halt

on seeing 'a' we are going to final state and whatever comes after it (including B) we are halting in final state

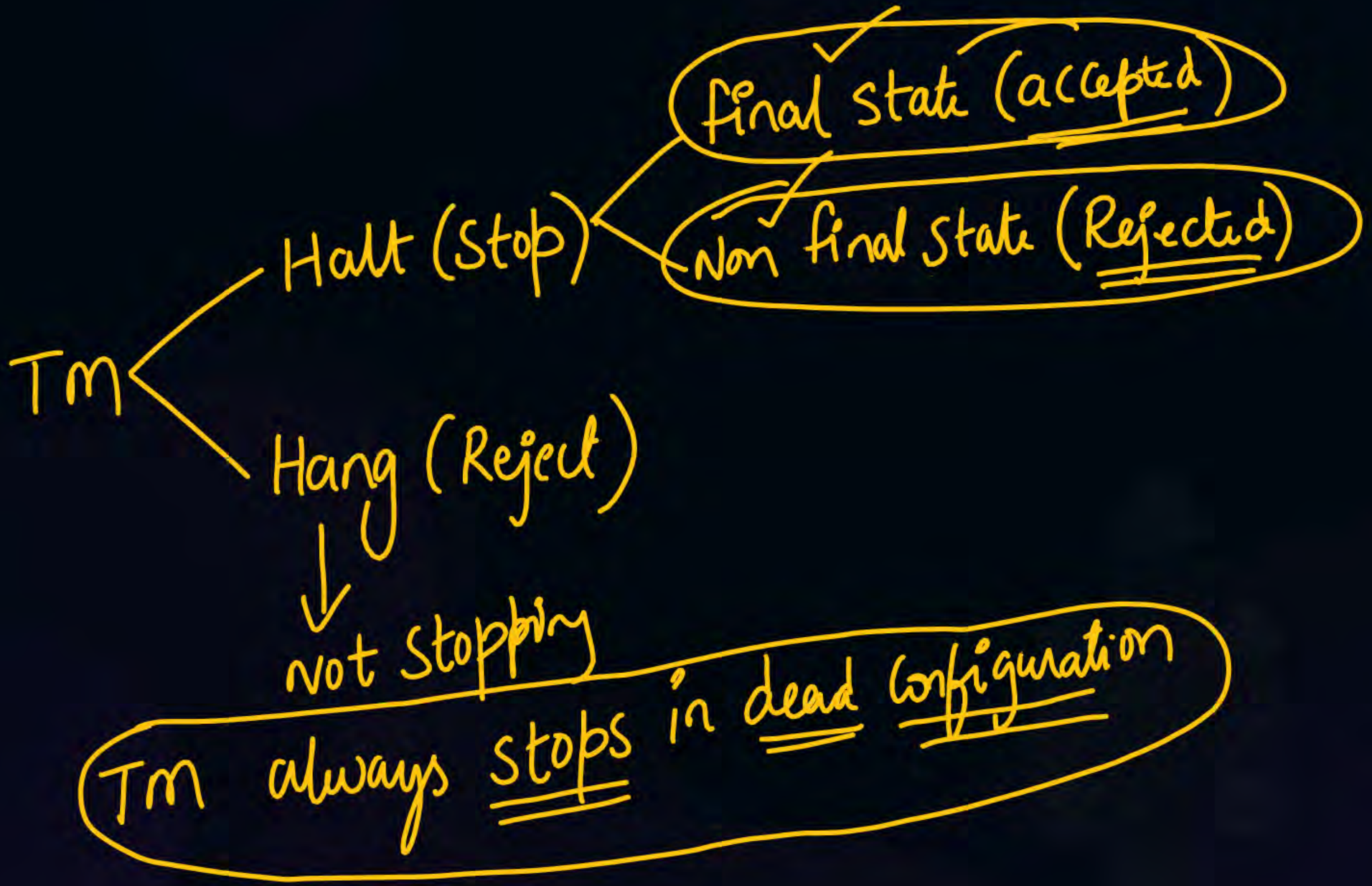
$a(a+b)^*$

ϵ → halt, Reject
a → halt, accept
b → halt, rejected
aa → halt, acc
ab → halt, acc
aaa → halt, acc

There is an algorithm to Convert

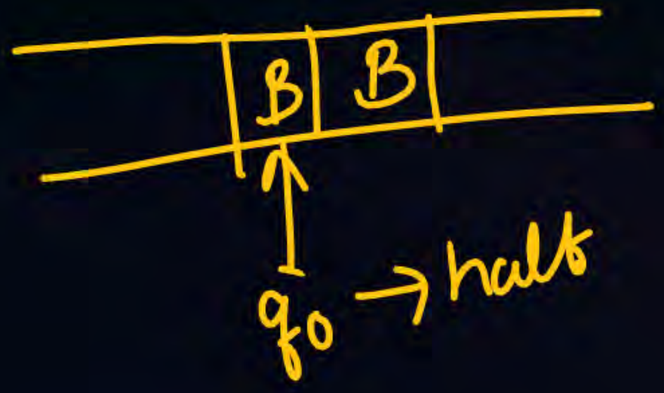
NTM → DTM

Power (NTM) ≅ Power (DTM)


TM
Halt (Stop)
final state (accepted)
Non final state (Rejected)
Hang (Reject)
not stopping
TM always stops in dead configuration



ϵ



ϵ - halt, rejected



ϵ

... B B a B B ...

q0 q1 (qf) → halt

ϵ - halt, rejected

a - accepted



ϵ

B B b B B

$q_0 \rightarrow$ halt

ϵ - halt, rejected

a - accepted, halt

b → halt, rejected



ϵ

B B a a B B

q_0 q_1 → halt



a á x
a b x
a á á x
a

ϵ - halt, rejected

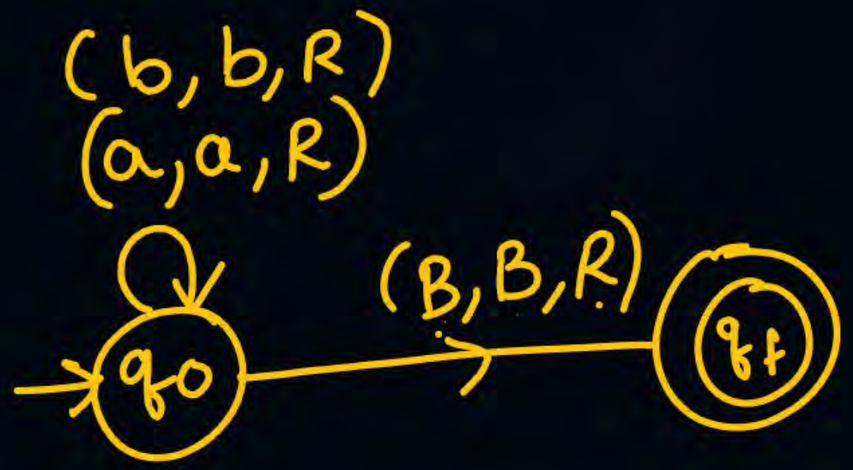
a - accepted, halt

b → halt, rejected

aa → halt, rejected

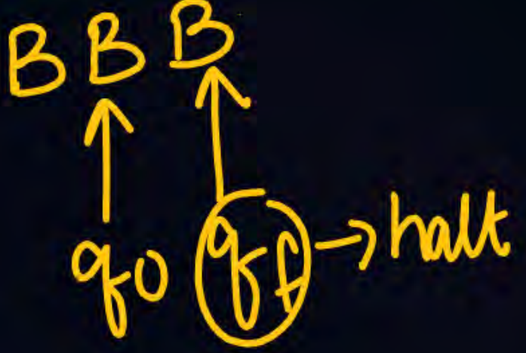
on seeing a 'a' we are going to q_1

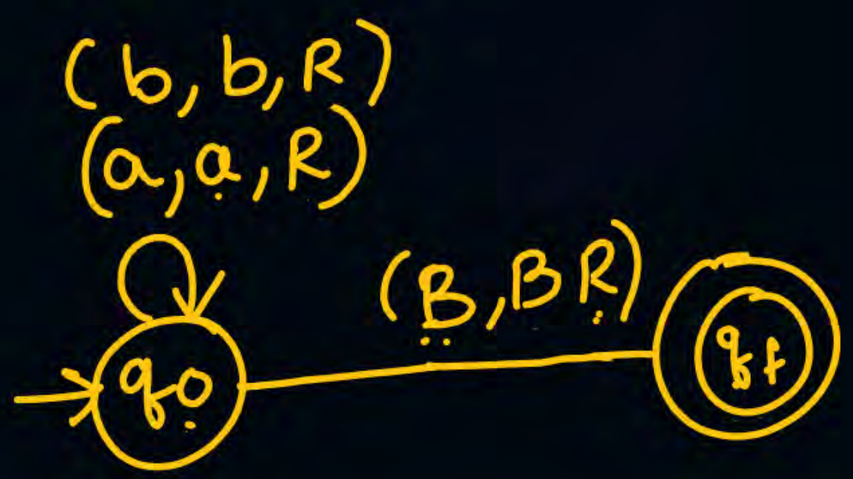
q_1 is going to q_f only upon seeing 'B'



$\epsilon \rightarrow$ halt, acc

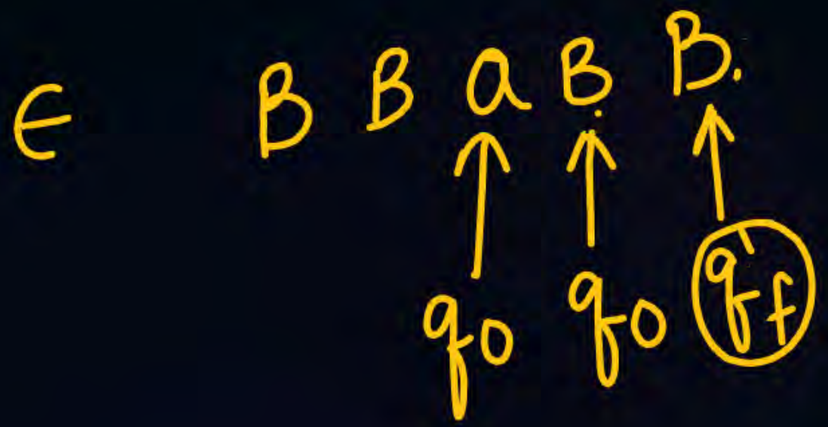
ϵ

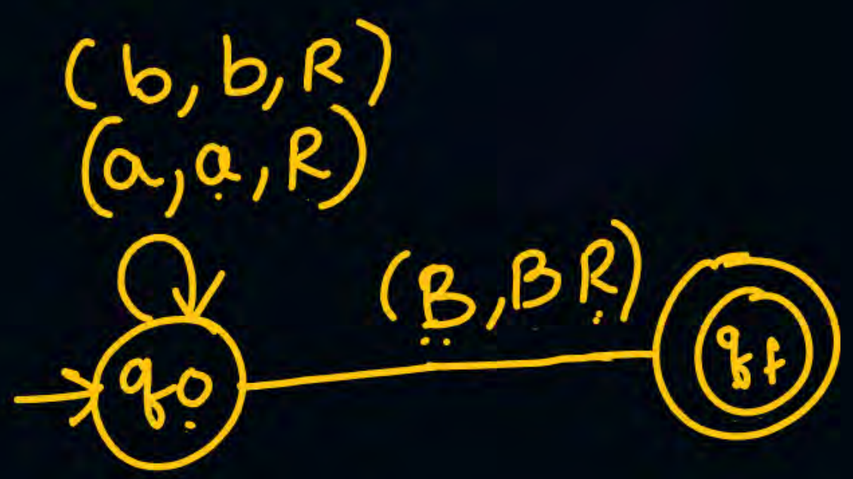




ϵ → halt, acc

a → halt, acc





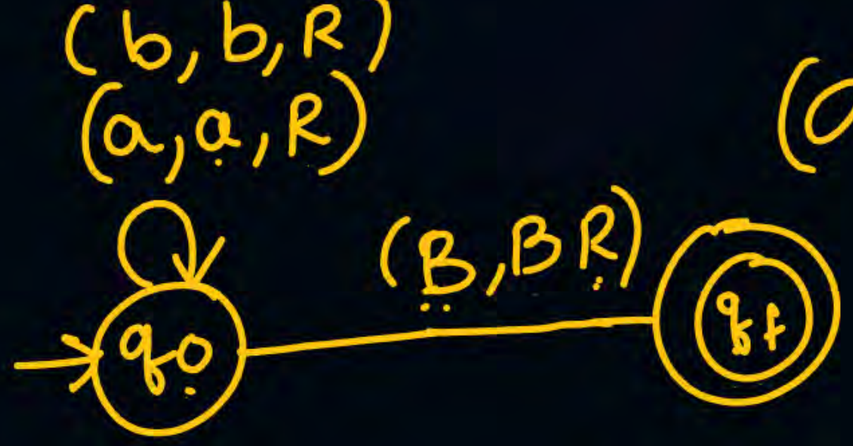
B B b B B

q0 q0 qf → halt

$\epsilon \rightarrow$ halt, acc

$a \rightarrow$ halt, acc

$b \rightarrow$ halt, acc



Halts on (a+b)* ✓
accepts (a+b)* ✓

ϵ → halt, acc

a → halt, acc

b → halt, acc

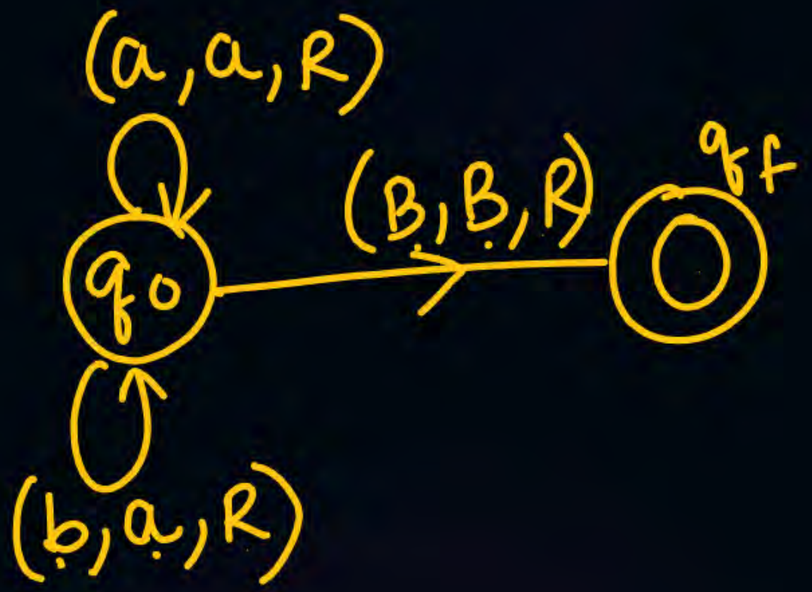
aa → halt, ac

a b a B B
↑ ↑ ↑ ↑ ↑
q0 q0 q0 q0 qf → halt

B B a a B B
↑ ↑ ↑ ↑ ↑ ↑
q0 q0 q0 qf → halt

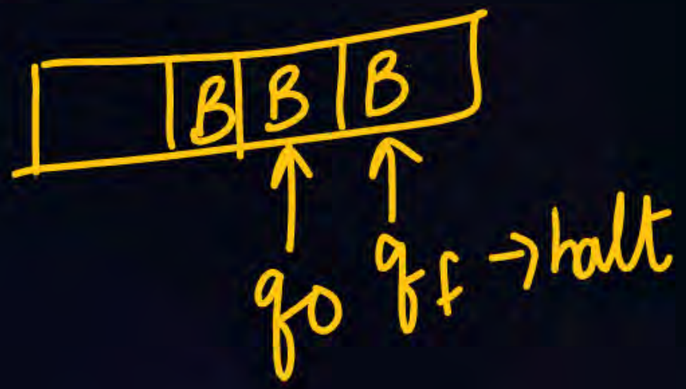
a b a B B
↑ ↑ ↑ ↑ ↑
q0 q0 q0 qf → halt

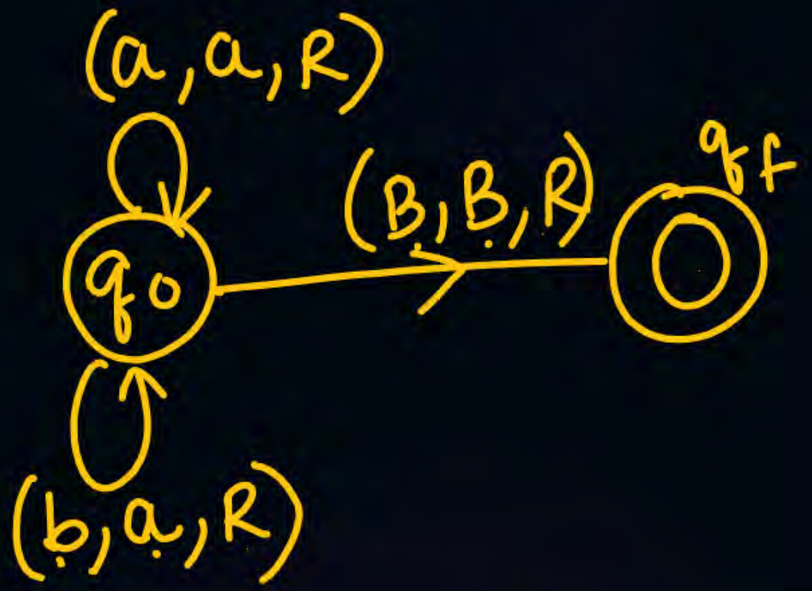
a a a B B
↑ ↑ ↑ ↑ ↑
q0 q0 q0 q0 qf → halt



$\in \rightarrow$ halt, acc

$\in \rightarrow$





$\in \rightarrow$ halt, acc

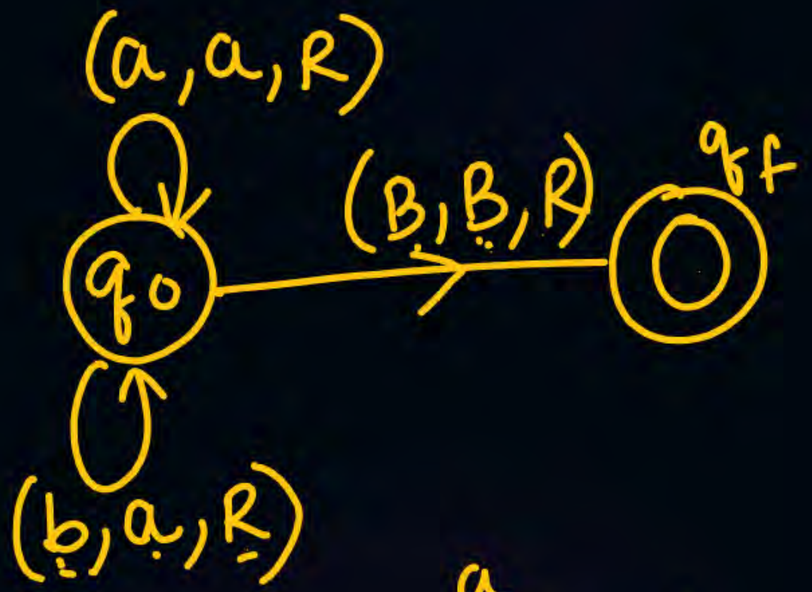
$a \rightarrow$ halt, acc

a

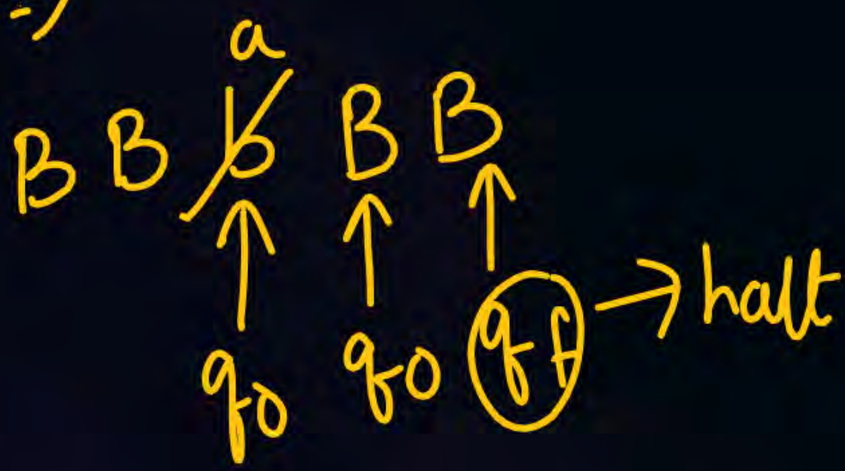
BB a B B

↑ ↑ ↑

q_0 q_0 q_f $\rightarrow$ halt



a

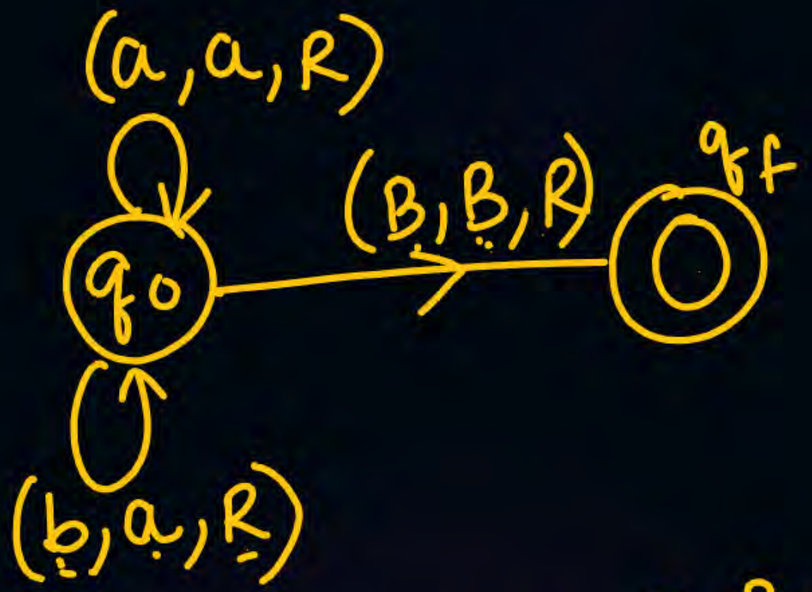


$\epsilon \rightarrow$ halt, acc

$a \rightarrow$ halt, acc

$b \rightarrow$ halt, acc

$b \rightarrow$ halt, acc



aa

B B a a B B

q_0 q_0 q_0 q_f → halt

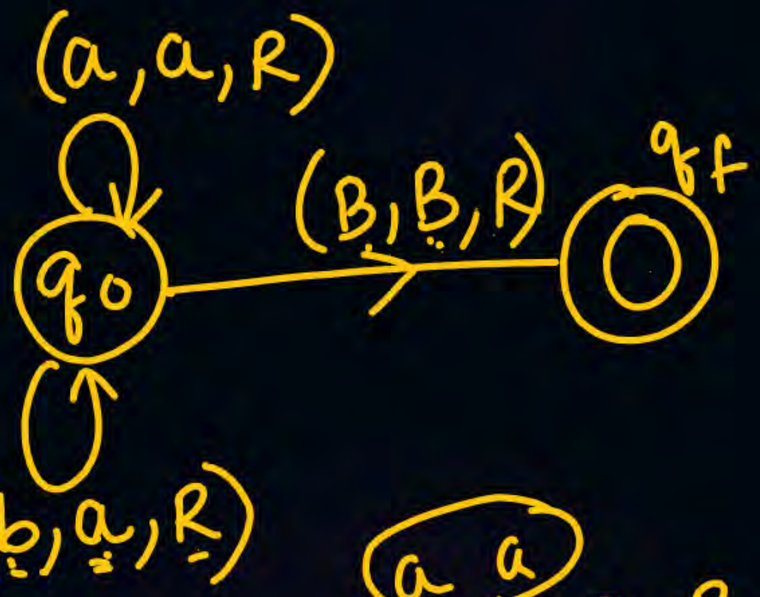
$\in$ → halt, acc

a → halt, acc

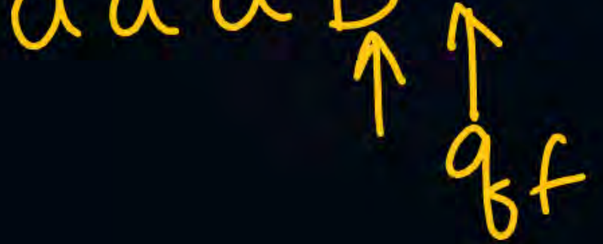
b → halt, acc

aa → halt, acc

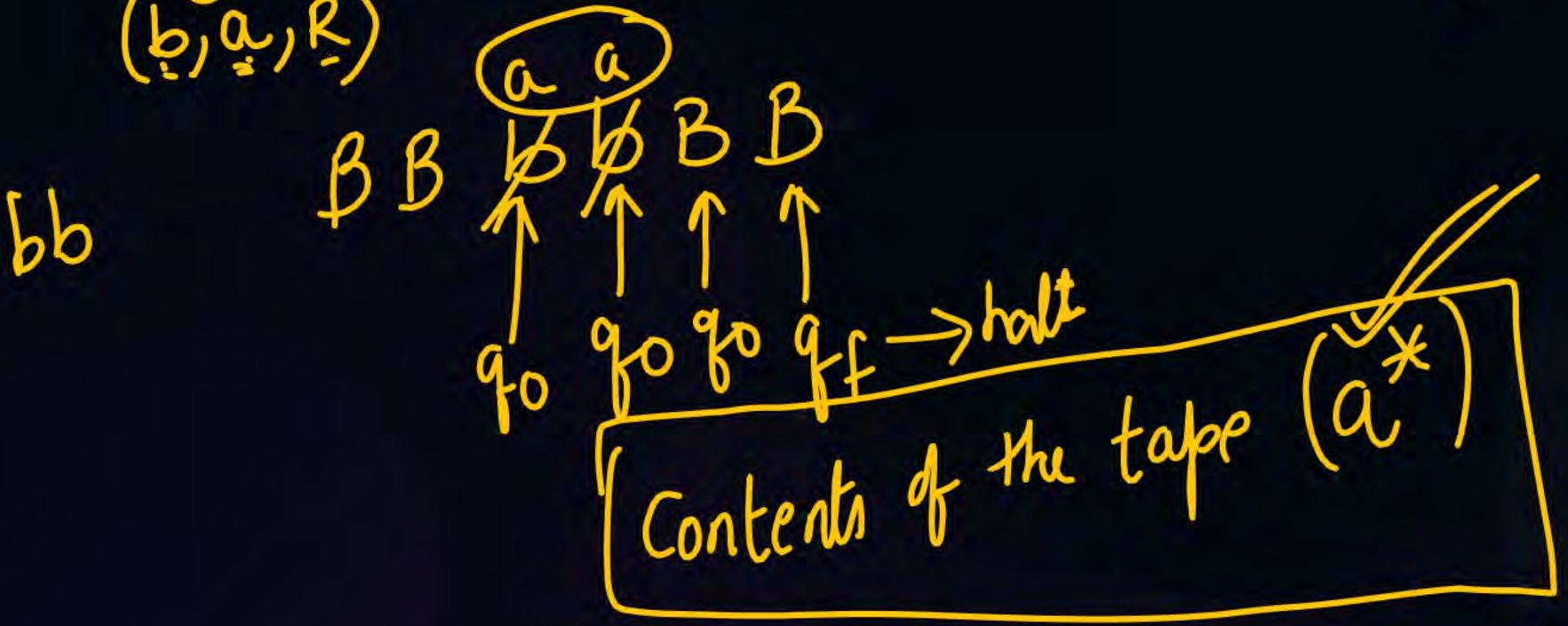
b → halt, acc

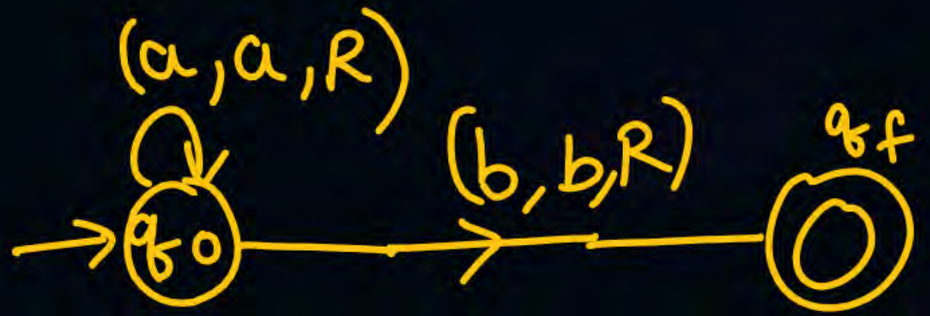


accepts $(a+b)^*$
halts on $(a+b)^*$

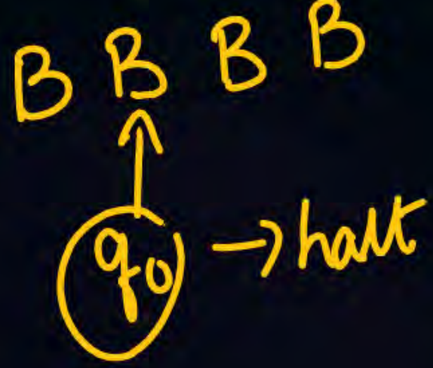


$\epsilon \rightarrow$ halt, acc
$a \rightarrow$ halt, acc
$b \rightarrow$ halt, acc
$aa \rightarrow$ halt, acc
$b \rightarrow$ halt, acc
$bb \rightarrow$ acc, halt

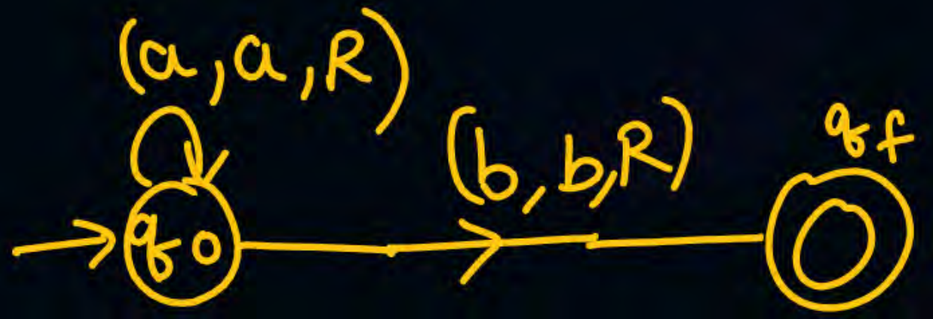




ϵ



ϵ - Halt, Rej



ϵ - Halt, Rej

a - Halt, Rej

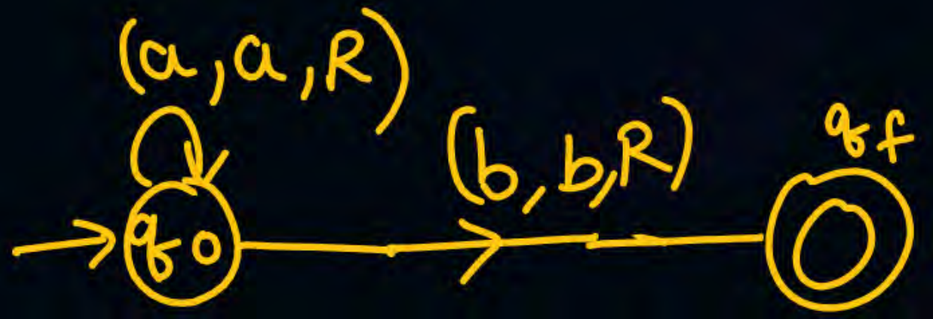
ϵ

a

B B a B B

↑ ↑

q_0 q_0 → halt



ϵ - Halt, Rej

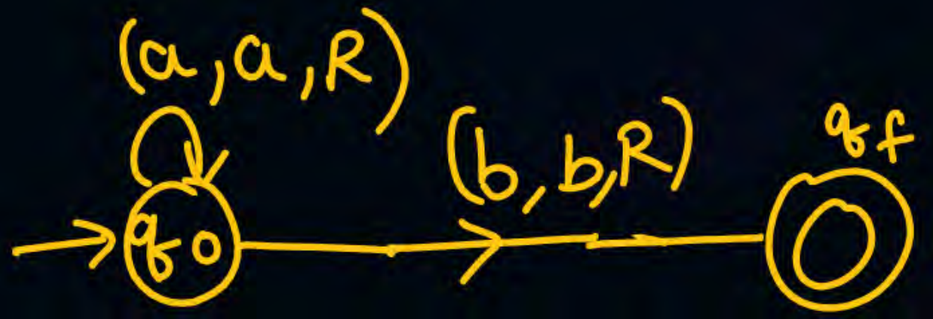
a - Halt, Rej

b - Halt, acc

b

BB b B B





ab

B B a b B B

q_0 q_0 q_f → halt

b

ab

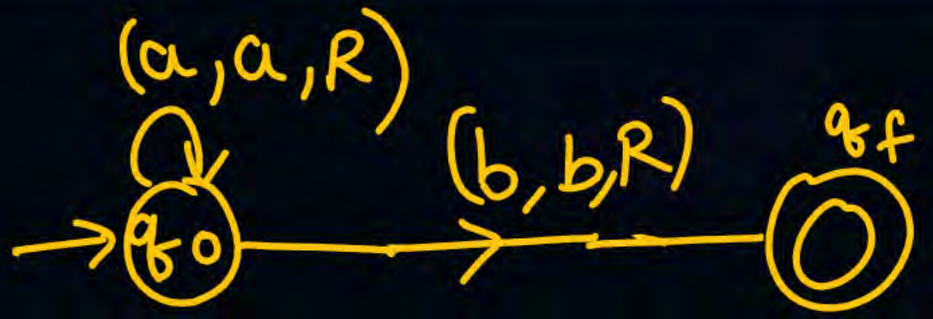
aab

$\in$ - Halt, Rej

a - Halt, Rej

b - Halt, acc

ab → halt, acc



BB a a B B

↑ ↑ ↑

q_0 q_0 q_0 → halt

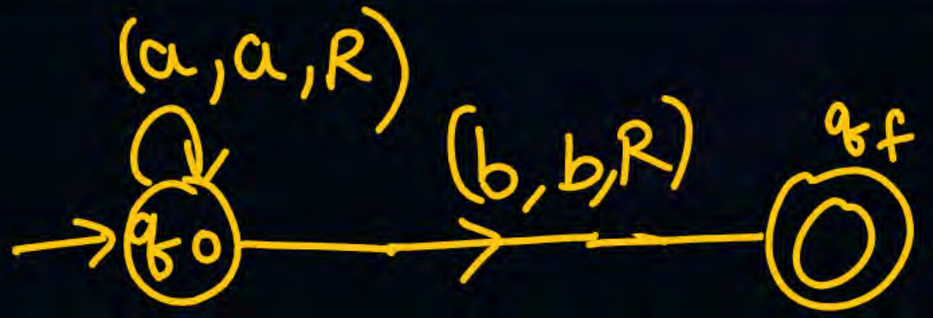
ϵ - Halt, Rej

a - Halt, Rej

b - Halt, acc

ab → halt, acc

aa → halt, Rej



B B b b B B

q_0 q_f → halt

ϵ - Halt, Rej

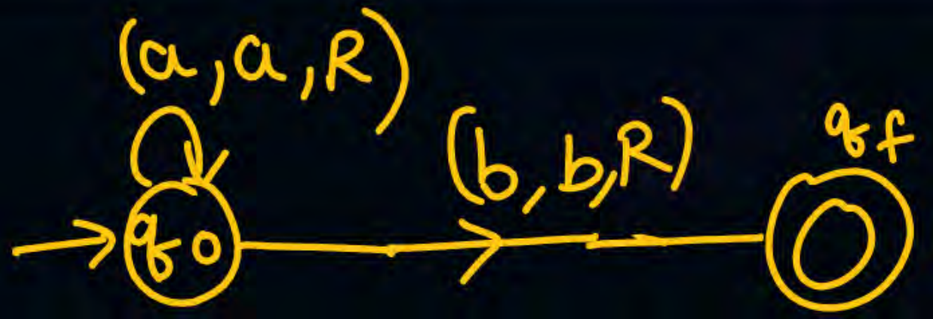
a - Halt, Rej

b - Halt, acc

ab → halt, acc

aa → halt, Rej

bb → halt, acc



B a a b B

q_0 q_0 q_0 q_f → halt

$a^*b(a+b)^*$

a ba

ϵ - Halt, Rej

a - Halt, Rej

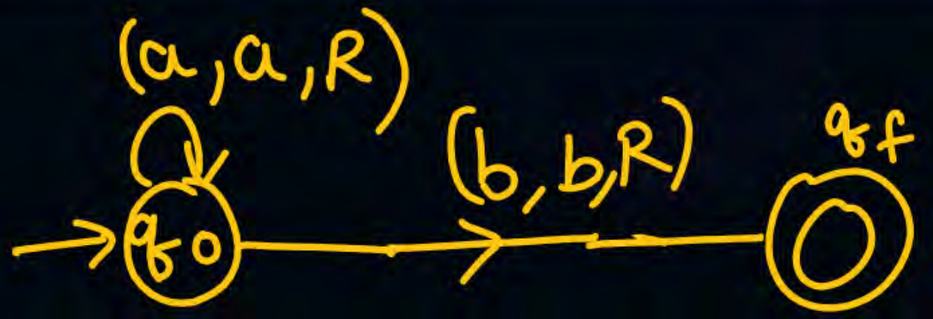
b - Halt, acc

ab → halt, acc

aa → halt, Rej

bb → halt, acc

aab → acc

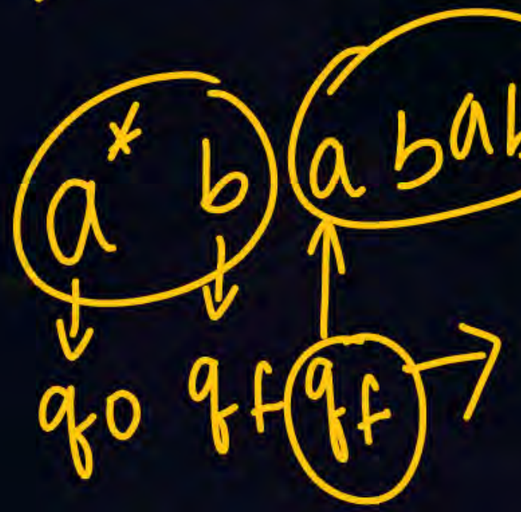


B B a b a B B

q_0 q_0 q_f → halt

$= (a+b)^* b (a+b)^*$

$a^* b (a+b)^*$



$\underline{a}^* \underline{b} (\underline{a+b})^*$

aba → H, acc

∈ - Halt, Rej

a - Halt, Rej

b - Halt, acc

ab → halt, acc

aa → halt, Rej

bb → halt, acc

aab → acc

$a^*b(a+b)^*$ → Containg at least one 'b'

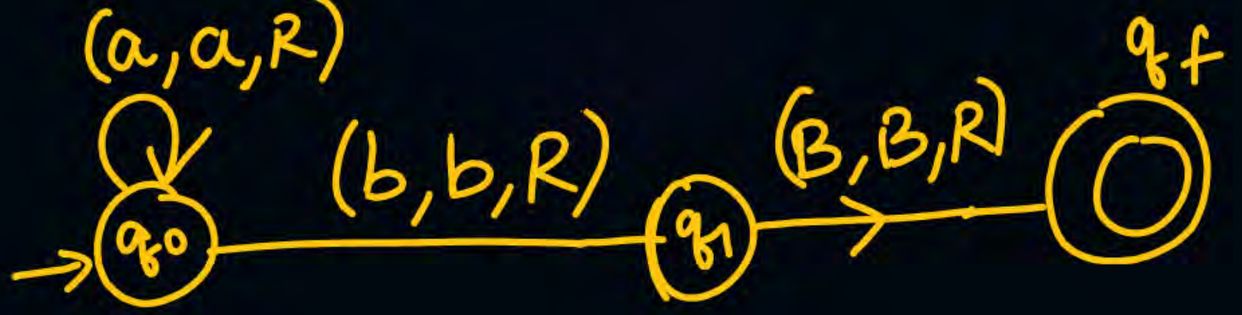


$a^*b(a+b)^* =$

$(a+b)^*ba^* = a^*b(a+b)^* = (a+b)^*b(a+b)^*$

Containg atleast one 'b'

$(a+b)^*b'(a+b)^*$

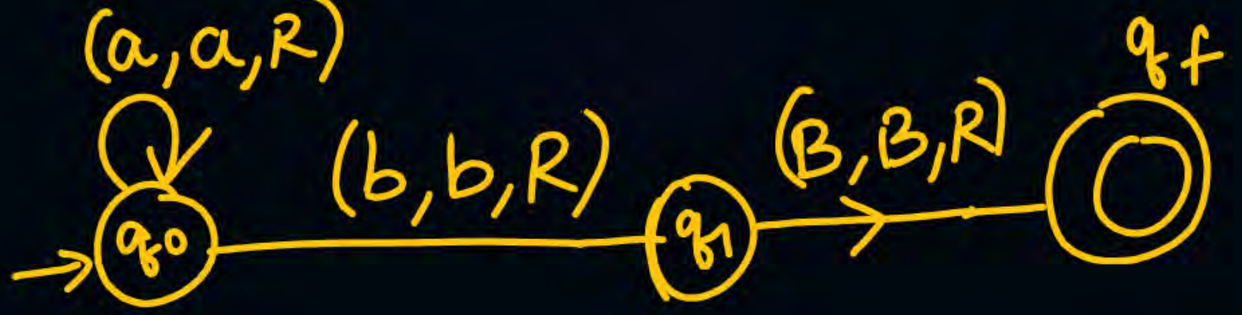


$a^{\neq}b$

b
a b
a a b
a b a
a b b

B B b B B
↑ ↑ ↑
q_0 q_1 q_f → halt

b → halt, acc

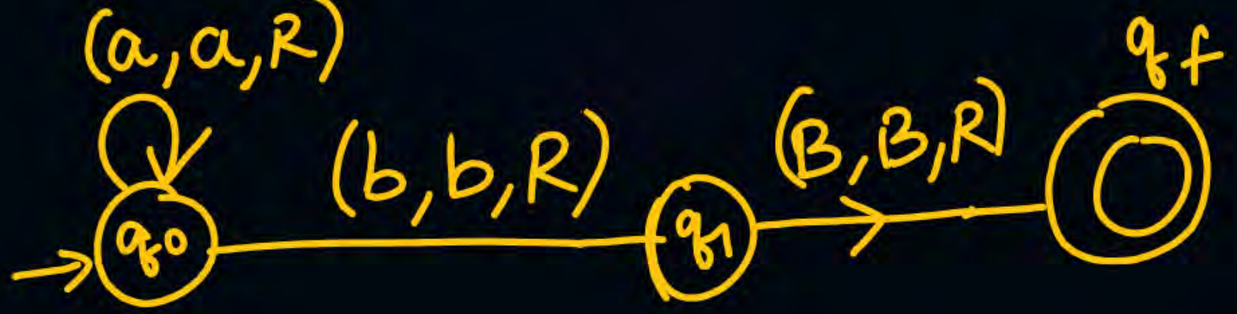


$a^{\neq}b$

b
ab
aab
abab
abb

B B a b B B
↑ ↑ ↑ ↑
q_0 q_0 q_1 q_f → halt

b → halt, acc
ab → halt, acc



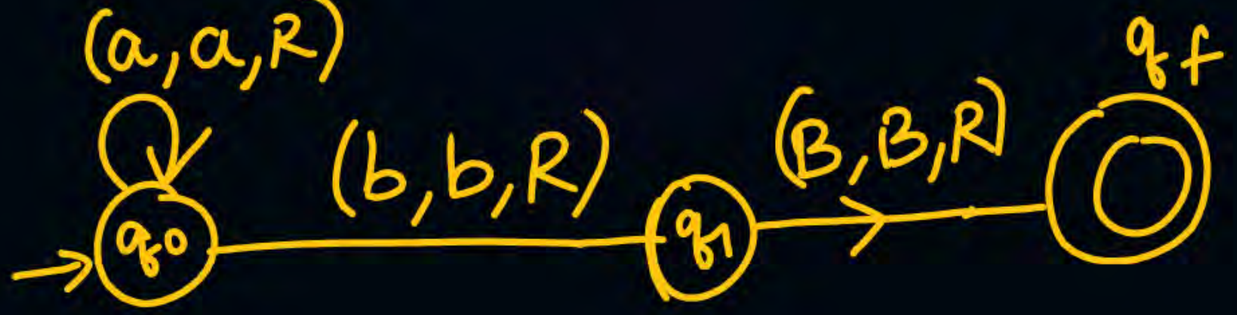
a^*b

b
ab
aab
abab
abb

a	a	b	B	B
↑	↑	↑	↑	↑
q_0	q_0	q_0	q_1	q_f → halt

a^*b( )

b → halt, acc
ab → halt, acc
aab → halt, acc



$a^* b$ ( )

b → halt, acc

ab → halt, acc

aab → halt, acc

aba → halt, Rej

$a^* b$

b

ab

aab

aba

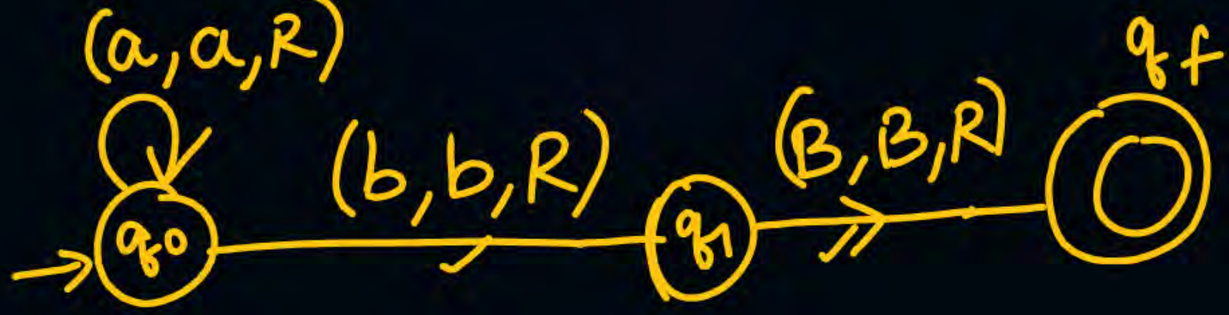
abb

B B a b a B B

↑ ↑ ↑

q_0 q_0 q_1 → halt

$a^x b$



halt on $(a+b)^*$

accepts a^*b

a^*b( )

a^*b

b

ab

aab

aba ✗

abb ✓

a b b B B

↑ ↑ ↑

q_0 q_0 q_1 → halt

a^*b

a^*ba ✗

b

b → halt, acc

ab → halt, acc

aab → halt, acc

aba → halt, acc

abb → halt, acc

TM with only right move,
accepts only regular languages



2 mins Summary

Topic Turing Machines

Topic

Topic

Topic

Topic

THANK - YOU